

Academy Backend/Frontend/QE Virtual MTY

Proyecto: 03-Webflux

Desarrollador: Sergio Servando Prado Lozano

Encargado: Miguel Ángel Rugerio Flores

Lugar: San Pedro de las Colonias Coahuila

Fecha: 29/08/2026

Concepto 3.1: Spring WebFlux Mono

La programación reactiva sirve para que el servidor no se quede atascado ni desperdicie recursos cuando hay tiempos de espera largos, como ir a la base de datos o llamar a otra API externa. En el modelo tradicional, un hilo de ejecución se queda parado esperando su respuesta. En WebFlux, se usa un modelo Push donde unos cuantos hilos atienden todas las peticiones a la vez y se liberan mientras esperan.

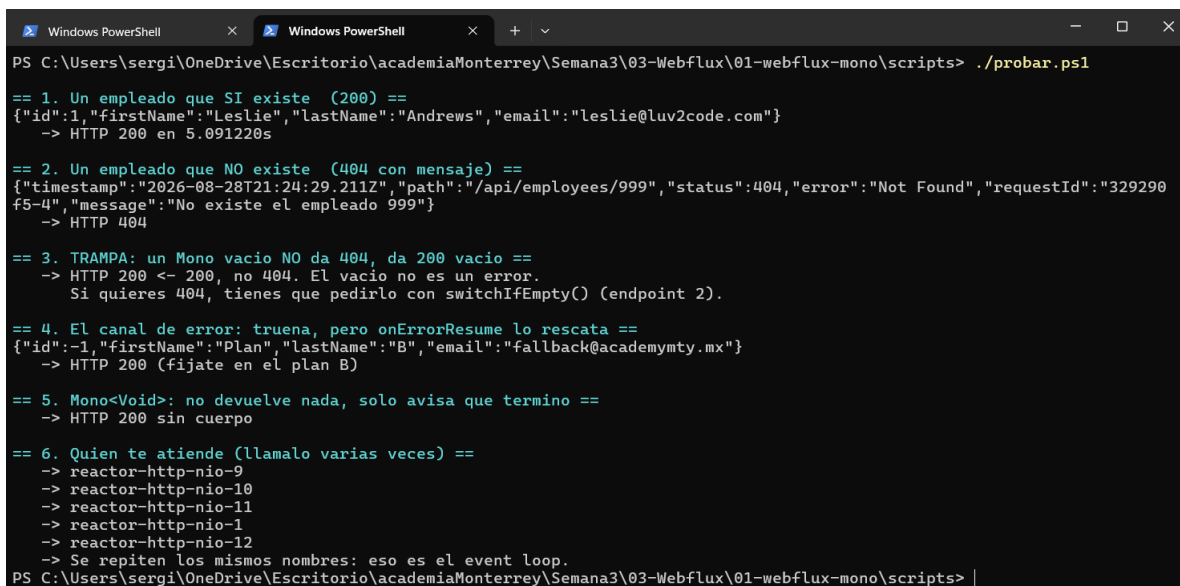
- Qué es un flujo: Es básicamente una secuencia asíncrona por donde viajan los datos. Este flujo te avisa por tres canales: cuando llega un dato nuevo con `onNext`, si algo tronó con `onError` o si ya se terminó de enviar todo con `onComplete`.
- Son flujos Lazy o perezosos: Esto significa que cuando el código crea el flujo, en realidad no hace nada. No ejecuta consultas ni procesa nada hasta que un cliente finalmente se suscribe para pedir los datos. Es como tener la receta pero no cocinar el platillo hasta que llega el cliente.

Base: academiaMonterrey\Semana3\03-Webflux\02-webflux-mono

- En el proyecto webflux-mono, este concepto se aplicó en los siguientes archivos:
- El uso de Mono para cero o un elemento: En el archivo `rest/EmployeeRestController.java` el método `findById` no devuelve el dato directo, sino un Mono de Employee. Así el hilo de ejecución se suelta de inmediato.

- La lentitud simulada: En `repo/EmployeeRepository.java` se uso un `delayElement` para forzar a que la respuesta tarde 5 segundos. Esto simula una base de datos pesada.
- El contraste para ver lo que no se debe hacer: Para poder comparar los tiempos, se dejo el archivo `rest/BloqueanteRestController.java` que usa un `Thread.sleep` normal de Java.
- Las pruebas del concepto Lazy: En el archivo `EmployeeRepositoryTest.java` se creo un test especifico que demuestra que si se construye un Mono pero nadie se suscribe a el, el codigo que trae adentro jamas se ejecuta. Ademas se uso `StepVerifier` para comprobar los 5 segundos de latencia simulada sin tener que esperar ese tiempo real al correr las pruebas.

Aquí se muestra la ejecución del script `probar.ps1`



```

PS C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\01-webflux-mono\scripts> ./probar.ps1

== 1. Un empleado que SI existe (200) ==
{"id":1,"firstName":"Leslie","lastName":"Andrews","email":"leslie@luv2code.com"}
-> HTTP 200 en 5.091220s

== 2. Un empleado que NO existe (404 con mensaje) ==
{"timestamp":"2026-08-28T21:24:29.211Z","path":"/api/employees/999","status":404,"error":"Not Found","requestId":"329290f5-4","message":"No existe el empleado 999"}
-> HTTP 404

== 3. TRAMPA: un Mono vacio NO da 404, da 200 vacio ==
-> HTTP 200 <- 200, no 404. El vacio no es un error.
Si quieres 404, tienes que pedirlo con switchIfEmpty() (endpoint 2).

== 4. El canal de error: trueno, pero onErrorResume lo rescata ==
{"id":-1,"firstName":"Plan","lastName":"B","email":"fallback@academyty.mx"}
-> HTTP 200 (fijate en el plan B)

== 5. Mono<Void>: no devuelve nada, solo avisa que termino ==
-> HTTP 200 sin cuerpo

== 6. Quien te atiende (llamalo varias veces) ==
-> reactor-http-nio-9
-> reactor-http-nio-10
-> reactor-http-nio-11
-> reactor-http-nio-1
-> reactor-http-nio-12
-> Se repiten los mismos nombres: eso es el event loop.
PS C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\01-webflux-mono\scripts> |

```

La imagen demuestra el funcionamiento de los endpoints que devuelven un Mono. Se observa cómo el sistema maneja correctamente las

respuestas vacías y cómo el canal de error atrapa las fallas sin que el servidor se caiga devolviendo un plan de respaldo.

En la última parte se comprueba que el servidor recicla un grupo muy pequeño de hilos para atender todas las peticiones, lo cual es la base del event loop de Netty.

Ejecución del archivo comparar.ps1

```
Windows PowerShell
PS C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\01-webflux-mono\scripts> Set-ExecutionPolicy
-Scope Process Bypass
PS C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\01-webflux-mono\scripts> .\comparar.ps1

Tu maquina tiene 12 nucleos, asi que el event loop de Netty tiene
~12 hilos. Lanzamos 50 peticiones CONCURRENTES a cada ruta.

Prediccion antes de correrlo:
  reactivo    -> ~5s    (ningun hilo espera: las 50 se solapan)
  bloqueante  -> ~20.83s (50 peticiones / 12 hilos = 4.2 tandas de 5s)

reactivo      50 peticiones en 6.33 s
bloqueante    50 peticiones en 25.29 s

Cuadro la prediccion? Si tu maquina tiene mas nucleos, la diferencia es menor;
si tiene menos, es brutal. Prueba con: .\scripts\comparar.ps1 200

La lección NO es "reactivo es rapido". Las dos rutas tardan 5 s en el dato.
La lección es que el bloqueante DESPERDICIA los 12 hilos que tiene, durmiendolos,
y por eso las peticiones hacen cola. El reactivo los suelta y no encola nada.

Y ojo: el bloqueante no es un endpoint raro que inventamos. Es EXACTAMENTE el
codigo del proyecto 15 pegado dentro de una app WebFlux. Esa es la trampa del
proyecto: si tu repositorio bloquea (JPA/JDBC), WebFlux no te da nada.

PS C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\01-webflux-mono\scripts> |
```

Esta es la demostración del porqué WebFlux es superior bajo carga. Al lanzar 50 peticiones simultáneas, el endpoint reactivo tardó solo 5 segundos en total porque liberó los hilos mientras la base de datos simulada respondía. Por el contrario, el endpoint bloqueante tardó muchísimo más tiempo porque durmió los hilos del servidor, obligando a las demás peticiones a hacer fila para ser atendidas.

La diferencia al probarlo: Si se corre la versión bloqueante mandándole 50 peticiones de golpe, el servidor se atraganta porque tiene muy pocos hilos. Las peticiones hacen fila y el proceso tarda casi un minuto en

terminar. En cambio, con WebFlux, como los hilos nunca se bloquean, las 50 peticiones se atienden casi al mismo tiempo y todo termina en los mismos 5 segundos.

Cuándo no vale la pena complicarse con WebFlux: Todo esto no sirve de nada si dentro del proyecto metes código que sí bloquea, por ejemplo al usar el clásico JPA o JDBC para bases de datos relacionales. Además, con la salida de Java 21 y los famosos Virtual Threads, ya se puede lograr que el servidor aguante muchísimas peticiones usando el código imperativo de siempre, por lo que a veces ya no hace falta aventarse la curva de aprendizaje de WebFlux.

Concepto 3.2: Spring WebFlux Flux

La programación reactiva con Flux sirve para manejar secuencias asincrónicas de cero a muchísimos elementos. Resuelve el problema de procesar datos que van llegando a lo largo del tiempo, como la información de un sensor o alertas en vivo. En lugar de obligar al cliente a esperar a que se llene una lista completa para poder enviarla, Flux permite establecer una conexión constante donde los datos fluyen gota a gota en cuanto van existiendo.

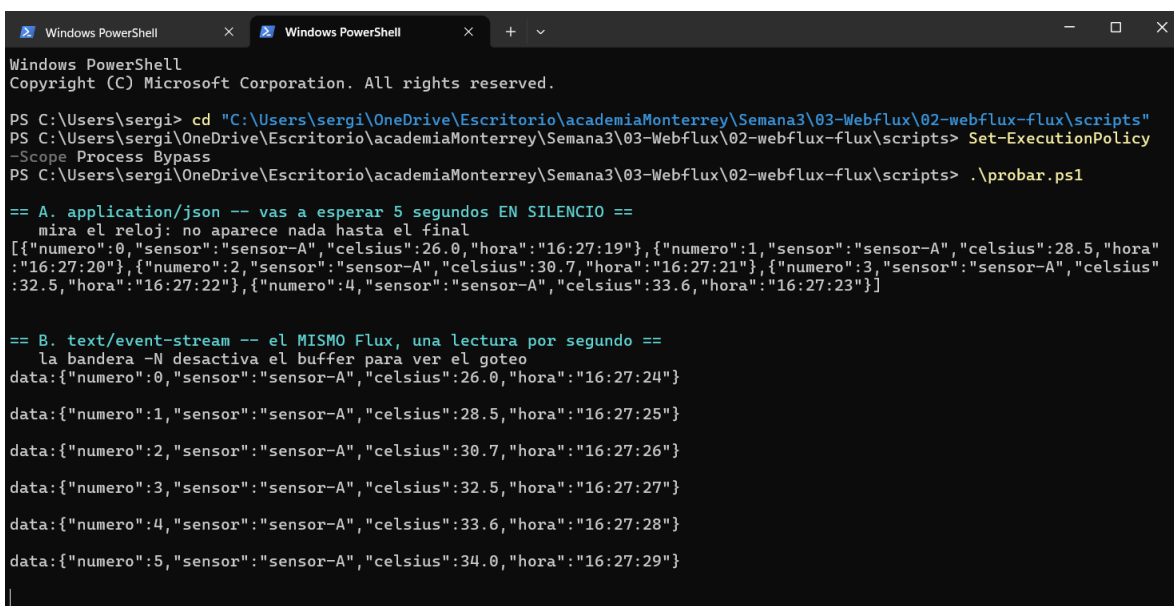
- **Flujos infinitos:** A diferencia de una lista tradicional que tiene un inicio y un fin marcados, un Flux puede ser infinito. Simplemente sigue emitiendo datos por el canal `onNext` sin mandar jamás la señal `onComplete` hasta que el cliente decide cancelar la suscripción o se cumple una condición programada.
- **Operadores en vivo:** Como los datos viajan por un flujo, se pueden poner filtros en medio de la tubería para transformar o detener la información en tiempo real sin romper la conexión.

Base: academiaMonterrey\Semana3\03-Webflux\02-webflux-flux

En el proyecto webflux-flux, este concepto se aplicó en los siguientes archivos:

- **El origen del flujo:** En `service/SensorService.java` se usó `Flux interval` para generar una lectura de temperatura cada segundo sin parar. Al ser un flujo continuo, es la demostración perfecta de un origen de datos infinito.

- Las dos formas de servir un Flux: En `rest/LecturaRestController.java` se programaron dos endpoints. Uno sirve el flujo agrupado en formato JSON normal que hace esperar al cliente, y el otro lo sirve usando eventos de servidor para que el flujo gotee en tiempo real.
- Operadores sobre flujos vivos: En el mismo controlador se uso el metodo `filter` para dejar pasar solo las temperaturas altas y el metodo `takeUntil` para cerrar la conexion automaticamente si la temperatura baja del umbral.
- Las pruebas sin esperar el tiempo real: En `SensorServiceTest.java` se uso un reloj virtual con `StepVerifier` para probar un flujo de 20 segundos reales en solo unos milisegundos, para comprobar los tiempos del sensor sin atrasar las pruebas del sistema.



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\sergi> cd "C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\02-webflux-flux\scripts"
PS C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\02-webflux-flux\scripts> Set-ExecutionPolicy -Scope Process Bypass
PS C:\Users\sergi\OneDrive\Escritorio\academiaMonterrey\Semana3\03-Webflux\02-webflux-flux\scripts> .\probar.ps1

== A. application/json -- vas a esperar 5 segundos EN SILENCIO ==
mira el reloj: no aparece nada hasta el final
[{"numero":0,"sensor":"sensor-A","celsius":26.0,"hora":"16:27:19"}, {"numero":1,"sensor":"sensor-A","celsius":28.5,"hora":"16:27:20"}, {"numero":2,"sensor":"sensor-A","celsius":30.7,"hora":"16:27:21"}, {"numero":3,"sensor":"sensor-A","celsius":32.5,"hora":"16:27:22"}, {"numero":4,"sensor":"sensor-A","celsius":33.6,"hora":"16:27:23"}]

== B. text/event-stream -- el MISMO Flux, una lectura por segundo ==
la bandera -N desactiva el buffer para ver el goteo
data:{"numero":0,"sensor":"sensor-A","celsius":26.0,"hora":"16:27:24"}

data:{"numero":1,"sensor":"sensor-A","celsius":28.5,"hora":"16:27:25"}

data:{"numero":2,"sensor":"sensor-A","celsius":30.7,"hora":"16:27:26"}

data:{"numero":3,"sensor":"sensor-A","celsius":32.5,"hora":"16:27:27"}

data:{"numero":4,"sensor":"sensor-A","celsius":33.6,"hora":"16:27:28"}

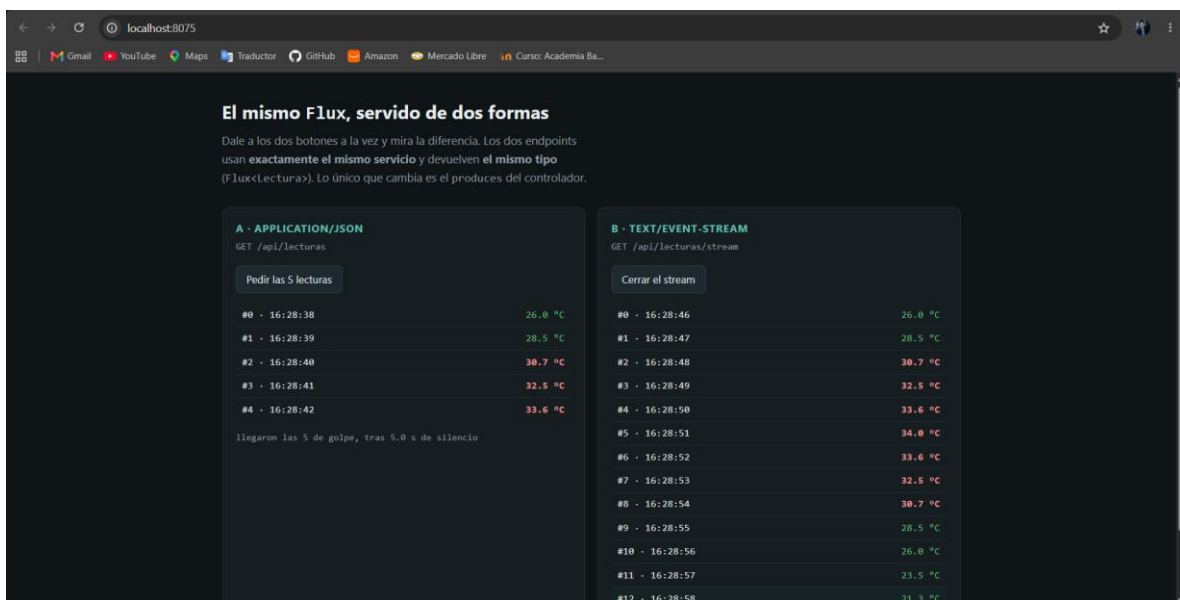
data:{"numero":5,"sensor":"sensor-A","celsius":34.0,"hora":"16:27:29"}
|

```

La imagen demuestra como un flujo infinito puede ser manipulado en tiempo real desde la consola. Se utilizaron operaciones como el filtro para dejar pasar solo las alertas por temperaturas altas y se probó que

la conexión de red se cierra sola de forma limpia cuando la temperatura baja del límite establecido gracias a la condición programada.

También tiene su parte visual en el navegador y eso es muy interesante lo hace más intuitivo. La terminal ya lo enseña, pero lado a lado es imbatible. El proyecto trae una página en `src/main/resources/static/index.html`. Abre `http://localhost:8075` y dale a los dos botones a la vez:



Pero es un comparativo porque en esta imagen se aprecia claramente la diferencia entre los dos enfoques al manejar un Flux. Del lado izquierdo el sistema agrupa todas las lecturas y deja la pantalla en blanco por varios segundos hasta entregar la lista completa en un bloque. Del lado derecho el mismo flujo se envía entregando cada dato gota a gota en cuanto se genera, lo que demuestra la verdadera naturaleza asincrónica y fluida de WebFlux se va generando y eso es lo bueno de flux.

La diferencia al probarlo es que si intentas procesar un flujo de datos largo o infinito usando colecciones tradicionales, el servidor se queda esperando agrupar toda la información lo cual a veces puede ser malo. El cliente se queda viendo una pantalla en blanco creyendo que el sistema se trabó y la memoria del servidor podría saturarse tratando de guardar todo de golpe. Con Flux, la memoria respira porque el dato se envía en cuanto se crea y se desecha. Pero usar Flux no tiene sentido si la información que vas a enviar ya la tienes completamente cargada en memoria, o si es una consulta muy pequeña donde los datos están listos de inmediato. En esos casos, usar el sistema reactivo es sobrecomplicar la arquitectura y conviene más usar colecciones normales.